GSoC'2026

OpenAstronomy

Organisation: OpenAstronomy

Sub Organisation: Radis

By: Merin Theres Jose (@merintheres)

Mentors: Nicolas Minesi (@minouHub), Pratham Hole (@Prtm2110)

Project Proposal:

Abstract:

RADIS is a high performance spectroscopic simulation library that operates on extremely large molecular databases such as HITRAN and HITEMP. As these datasets grow to tens of gigabytes, efficient data loading, storage, and memory management become critical challenges. The current system relies on HDF5 and Vaex for lazy loading; however, Vaex is no longer actively maintained, raising concerns about long term sustainability.

This project proposes a modern data processing architecture based on Parquet as the primary cache format and a pluggable backend system supporting modern DataFrame engines. Benchmarking on real and synthetic CO_2 datasets shows that both Polars and DuckDB provide strong performance and memory efficiency, making them suitable replacements for Vaex.

While both engines perform well, Polars integrates naturally with the existing pipeline and allows largely compatible mapping for common operations, reducing migration complexity. Therefore, a single backend will be selected for implementation, while keeping the architecture flexible for future extensions. Additionally, a configurable LRU cache management system will be introduced to control disk usage, improving overall performance and maintainability.

Problem Statement:

Modern spectroscopic databases such as HITEMP can exceed tens of gigabytes and contain millions to billions of spectral lines. Efficiently loading and processing this data is a major challenge in RADIS.

The current approach relies on HDF5 storage with Vaex for lazy loading. However, Vaex is no longer actively maintained, raising concerns about long-term sustainability and compatibility. Pandas based approaches require loading large datasets into memory, often resulting in memory exhaustion.

Additionally, HDF5 limits interoperability and compression efficiency compared to modern columnar formats such as Parquet.

A scalable, maintainable, and memory efficient solution is required to enable RADIS to handle large spectroscopic datasets reliably while remaining future proof.

CURRENT SYSTEM:

RADIS processes spectroscopic databases by converting compressed files into an HDF5-based format for faster reuse. These HDF5 files store large DataFrames containing molecular line parameters, which are later used for spectral computations.

To handle large datasets, RADIS integrates Vaex for memory mapped, lazy loading of HDF5 files, allowing partial access to data without loading everything into memory. In cases where Vaex is not used, Pandas based loading is employed, which results in full data being loaded into memory.

While this approach improves performance compared to repeated parsing, it introduces several limitations. The reliance on Vaex creates maintainability concerns due to its lack of active development, and Pandas based workflows are not scalable for very large datasets due to high memory usage. In practice, loading large HITEMP datasets (e.g., CO_2 over wide spectral ranges) using Pandas often leads to memory exhaustion and crashes on standard machines , making such workflows unreliable for real use cases. Additionally, the current system tightly couples the storage format and DataFrame engine, limiting flexibility for future improvements.

IMPLEMENTATION PLAN

Phase 1: Benchmark Validation and Design

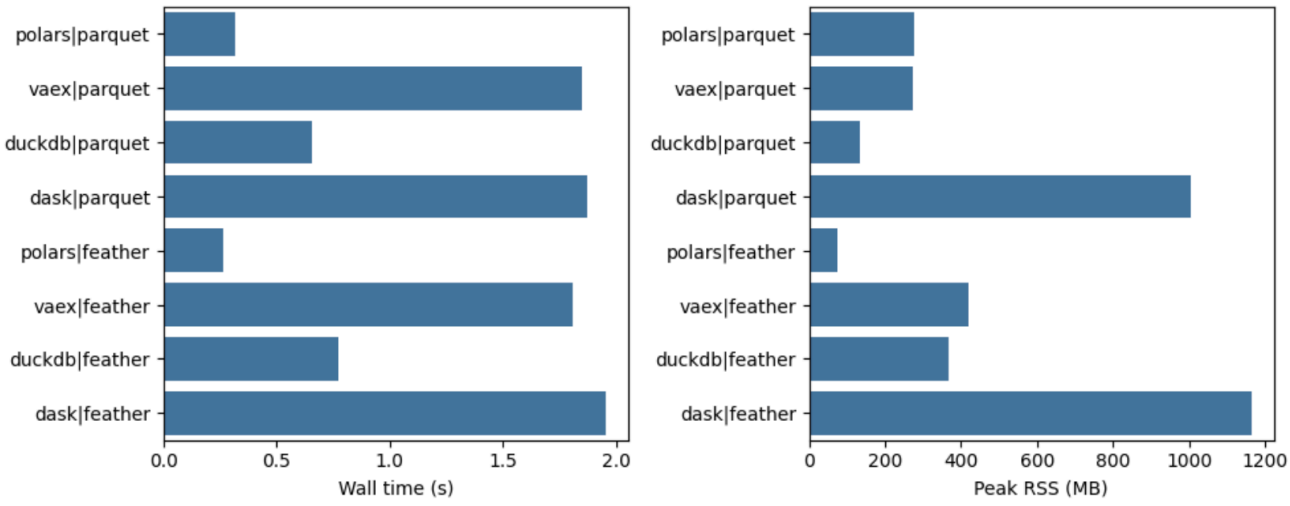
Validate benchmarking results and analyze the RADIS data pipeline to identify integration points. Benchmark scripts, datasets, and results are in:

https://github.com/merintheres/radis_proposal

Key modules such as radis/api/hdf5.py, dbmanager.py, and hitempapi.py will be analyzed to identify all engine dependent code paths

BENCHMARKING AND ANALYSIS:

- A systematic benchmarking study was conducted to evaluate both DataFrame engines (Polars, DuckDB, Dask, Vaex) and storage formats (Parquet and Feather) as modern alternatives to the current Vaex + HDF5 pipeline.
- Experiments were performed on:
    1. Real dataset: CO_2 HITEMP data (wavenumber range: 500–1000 cm^{-1}).
    2. Synthetic dataset: Randomly generated dataset with a similar schema for controlled evaluation
- All experiments were conducted on a MacBook Air M4 with 16 GB RAM.
- Each engine + format combination (e.g., Polars + Parquet, DuckDB + Feather) was executed in an isolated subprocess to ensure fair comparison. Wall time was measured for end to end execution, while peak memory usage (RSS) was tracked by periodically sampling the process during runtime.
- This setup allowed evaluation of:
    1. Execution performance across engines
    2. Memory efficiency under realistic workloads
    3. Storage performance trade offs between Parquet and Feather



Results Summary:

- Polars consistently achieves the lowest execution time across workloads
- DuckDB provides comparable performance with efficient memory usage
- Dask introduces overhead for single-node workloads
- Vaex shows inconsistent performance and is no longer maintained

From a storage perspective:

- Parquet provides significantly better compression and scalability
- Feather offers slightly faster execution in some cases but at a much larger file size

Observed sizes for the same dataset:

- Feather: ~5.4 GB
- Parquet: ~1.03 GB

Overall, Polars and DuckDB emerge as strong engine candidates, while Parquet is identified as the most practical storage format due to its balance of performance, compression, and interoperability. Polars was also evaluated for API compatibility with Vaex to ensure minimal migration complexity.

Detailed benchmark code and reproducible results:

https://github.com/merintheres/radis_proposal

Phase 2: DataFrame Adapter Implementation

Introduce a backend abstraction layer to replace existing engine-specific if/else logic with a unified interface. A prototype implementation is available here:

https://github.com/merintheres/radis_proposal/tree/main/dataframe_backend

Currently, RADIS uses engine-based branching such as:

```
if engine == 'vaex':
    df = vaex.open(path)
elif engine == 'pandas':
    df = pd.read_hdf(path)
else:
    raise ValueError
```

This will be replaced with a backend adapter:

```
adapter = DataFrameAdapter(backend=engine)
df = adapter.load(path, columns=columns)
```

The adapter will encapsulate operations such as loading, filtering, and column selection, and will be integrated into core components like DataFileManager (radis/api/hdf5.py), removing scattered engine-specific logic and enabling integration of the selected backend (e.g., Polars) while keeping the architecture extensible for future engines such as DuckDB..

Similar branching exists inside DataFileManager:

```
if self.engine == "pytables":
    ...
elif self.engine == "vaex":
    ...
else:
    raise NotImplementedError
```

Similarly, this will be refactored using backend delegation:

```
self.backend = get_backend(engine)
return self.backend.write(...)
```

This change will be applied across data loading and writing functions (e.g., DataFileManager in radis/api/hdf5.py), centralizing backend logic and enabling clean integration of Polars.

Phase 3: Polars Integration and Migration Strategy

Following the abstraction layer, the selected backend (Polars) will be integrated by systematically replacing Vaex-based operations across the I/O and computation layers, while preserving the existing user-facing API.

The external API (e.g., calc_spectrum()) will remain unchanged; all modifications are internal to the data loading and computation pipeline

API Mapping Strategy

The migration is simplified due to similarities between Vaex and Polars APIs:

1. `vaex.open()` → `pl.scan_parquet()`
2. `vaex.concat()` → `pl.concat()` or `pl.scan_parquet(list_of_files)`
3. `vaex.from_csv()` → `pl.scan_csv()`
4. `vaex.from_pandas()` → `pl.from_pandas()`
5. `vaex.vconstant()` → `pl.lit()`
6. `df.mean(..., binby=...)` → `group_by().agg()`

This ensures a direct and predictable mapping of operations with minimal structural changes.

I/O Layer Refactoring

Refactoring will be applied across key modules to replace Vaex-based loading with Polars lazy execution:

1. Core I/O Integration (radis/api/hdf5.py)
   Extend DataFileManager to support a Polars backend with Parquet read/write support.
2. Multi-File Loading (radis/api/dbmanager.py)
   Replace vaex.open() with pl.scan_parquet() for lazy multi-file loading.
3. API Parsing (radis/api/hitempapi.py, exomolapi.py)
   Replace CSV and chunk-loading logic:
   a. vaex.from_csv() → pl.scan_csv()
   b. vaex.concat() → lazy multi-file scanning

Example (HITEMP chunk loading):

```
# BEFORE
df = vaex.concat([vaex.open(f) for f in chunk_files])
# AFTER
lf = pl.scan_parquet(chunk_files)
df = lf.filter(pl.col("wav").is_between(wmin, wmax)).collect()
```

4. Cache System (radis/api/cache_files.py)
   Register Polars as a valid backend and update cache detection logic to handle Parquet files.

Computation Layer Migration

Vaex-specific computation logic will be replaced with Polars native expressions:

Constant column creation:

```
# BEFORE
df["col"] = vaex.vconstant(...)
# AFTER
df = df.with_columns(pl.lit(...).alias("col"))
```

Binning and aggregation:

```
# BEFORE
df.mean("gj", binby="Erot")
# AFTER
df.group_by("bin").agg(pl.col("gj").mean())
```

These changes will be applied across:

- `radis/lbl/base.py`
- `radis/lbl/loader.py`
- `radis/levels/partfunc.py`

This ensures a complete backend transition across both data access and computation layers.

Phase 4: Parquet Cache Integration

The pipeline will be extended to support Parquet as the primary cache format, replacing HDF5 while ensuring backward compatibility and safe data handling.

Unlike HDF5, Parquet does not support in-place appends. Therefore, cache generation will follow a partitioned / chunk-based design rather than modifying a single file.

Append Strategy and Dataset Layout:

Instead of appending to a single file, data will be written as multiple Parquet parts:

```
cache_dir/
    dataset.parquet/
        Part-0001.parquet
        Part-0002.parquet
        ...
        _manifest.json
```

This preserves append semantics while remaining compatible with Parquet's immutable design.

Atomic Writes and Safety:

To avoid partial writes or corruption:

- Data will be written to temporary files
- Final files will be moved using atomic rename (os.replace)

```
tmp = path + ".tmp.<uuid>"
write(tmp)
os.replace(tmp, path)
```

This guarantees consistency even in case of crashes or interruptions.

Metadata Handling:

Since HDF5 attributes are not directly portable, metadata will be stored in two forms:

- Parquet key-value metadata (via PyArrow)
- A sidecar JSON file (e.g., dataset.meta.json)

Stored metadata includes:

- wavenumber range (min/max)

- molecule / isotope information

- row counts

- RADIS version

This ensures compatibility, reproducibility, and efficient dataset reconstruction.

Cache Detection and Compatibility:

Cache detection will not rely solely on file extensions. Instead, a robust detection strategy will be used:

```
def detect_cache_engine(path):
    try:
        import pyarrow.parquet as pq
        pq.ParquetFile(path)
        return "polars"
    except:
        try:
            import tables
            tables.open_file(path)
            return detect_hdf5_engine(path)
        except:
            return "unknown"
```

A higher-level utility will use this detection:

```
df = smart_read_dataframe(path)
```

This allows seamless interoperability between HDF5 and Parquet caches without requiring user-side changes.

Integration with Existing Pipeline:

- DataFileManager will delegate read/write operations to the backend

- Parsers (e.g., hitempapi.py) will continue using append-style writes

- Backend will internally handle partitioning and manifest updates

Concurrency and Consistency:

To handle concurrent access safely:

- Writes will use atomic rename
- Manifest updates will be serialized (or optionally protected via file locks)
- Partial writes will not be exposed to readers

This ensures robustness in multi-process or interrupted execution scenarios.

Phase 5: Caching and Optimization

Integrate an optional LRU-based cache manager to control disk usage using metadata such as file size and last access time. This follows the prototype implementation and discussion here:

https://github.com/merintheres/radis_proposal/tree/main/lru_cache

https://github.com/radis/radis/issues/962

The CacheManager maintains metadata for each cache file (path, size_mb, last_used) and evicts least recently used files when a configurable storage limit is exceeded:

```
class CacheManager:
    def evict_lru(self, limit_mb):
        files = sort_by_last_used(self.files_meta)
        while total_size(files) > limit_mb:
            remove(files.pop(0))
```

The manager will integrate with the RADIS pipeline by updating access metadata during read/write operations:

```
def read(self, path, ...):
    self.cache_manager.update_last_used(path)
    return self.backend.read(path, ...)
```

This system is designed as an optional cleanup utility, ensuring that cache eviction only occurs when explicitly triggered, avoiding unintended data loss during normal workflows.

Phase 6: Testing and Documentation

Develop unit and integration tests to validate correctness and performance, and document the new backend system, cache behavior, and usage.

Tests will include backend round-trip validation (write → read → compare), cache format detection checks, and verification that Parquet and HDF5 pipelines produce consistent outputs across workflows.

Phase 7: Final Benchmarking

Perform large-scale benchmarking on HITEMP CO_2 datasets to validate improvements and extend the implementation to other databanks and clients within RADIS.

Benchmarks will measure execution time, peak memory usage, and cache size, comparing the new engine + Parquet pipeline with the existing system to quantify performance gains.

TECHNICAL APPROACH:

- A common interface will be introduced to standardize operations such as: Data loading, Filtering, Column selection and Aggregation
- One backend will be implemented as the primary execution engine.
    1. Likely choice: Polars (due to ease of integration with Python workflows)
    2. Supports lazy execution using scan_parquet
    3. Enables optimized query execution through predicate pushdown and column pruning
- Storage Transition
    1. Introduce Parquet as the primary cache format
    2. Modify parsing pipelines to generate Parquet caches
    3. Provide migration tools for existing HDF5 caches
- Cache Management
    1. Implement LRU eviction using metadata (`last_used`, `size`)
    2. Ensure user controlled cleanup to prevent unintended data loss

PROPOSED SOLUTION:

Based on benchmarking results and repository analysis, this project proposes replacing the current Vaex + HDF5 workflow with a Parquet-based caching system and a pluggable DataFrame backend. A backend abstraction layer will be introduced in DataFileManager, allowing integration of a selected modern engine. Based on integration ease and performance, a primary backend will be implemented (with Polars as the most suitable candidate), enabling efficient data loading, filtering, and computation using lazy execution.

To support this transition, HITEMP parsing pipelines will be modified to generate Parquet-based caches instead of HDF5. Existing HDF5 caches will be handled through an automatic conversion mechanism within the pipeline, ensuring a transition without requiring manual intervention from users. Additionally, a configurable LRU cache management system will be introduced to control disk usage by evicting least-recently-used files based on metadata such as size and last access time.

Overall, this approach improves performance, memory efficiency, and maintainability while providing a flexible foundation for future extensions.

TIMELINE:

PHASE		
1	**Week 1-2**	- Validate benchmarks and analyze RADIS data pipeline - Identify engine-dependent code paths and finalize backend design
2	**Week 3-4**	- Implement backend abstraction and integrate with DataFileManager - Replace engine-specific logic with backend delegation
3	**Week 5-6**	- Integrate Polars backend for data loading and lazy execution - Refactor I/O modules for multi-file and parser support
4	**Week 7-8**	- Implement Parquet-based cache system (partitioned layout + manifest) - Add atomic writes, metadata handling, and cache detection
3	**Week 9-10**	- Migrate computation layer from Vaex to Polars - Ensure numerical consistency and introduce compatibility shims
5	**Week 11**	- Implement LRU cache manager with metadata tracking and safe eviction
6-7	**Week 12**	- Perform testing, benchmarking, and final integration - Complete documentation and code cleanup

EXPECTED OUTCOME:

The project will result in a scalable and maintainable data processing pipeline for RADIS, capable of efficiently handling large spectroscopic datasets. Key outcomes include:

- Integration of a modern Polars based backend replacing Vaex.
- Improved memory efficiency and performance for large datasets.
- Implementation of lazy loading mechanisms for HITEMP databases.
- Parquet as the primary cache format with backward compatibility for HDF5
- Comprehensive benchmarking report comparing different approaches.
- A fully tested and documented implementation ready for production use.

After Summer of Code:

After GSoC, I plan to continue contributing to RADIS by maintaining and improving the backend system, optimizing performance, and supporting future extensions such as additional engine integrations.

I intend to remain actively involved in the RADIS community and contribute to future improvements in the codebase.

Development Experience:

I have been actively contributing to RADIS, where I have worked on improving database handling, optimizing workflows, and refining existing features. Through these contributions, I have developed a strong understanding of the RADIS codebase, particularly its data pipeline and database management system.

Alongside my open source work, I have built several Python based projects during my second year of engineering, focusing on problem solving, data handling, and performance oriented implementations. These experiences have strengthened my ability to work with large datasets and write efficient, maintainable code.

I have also contributed to other open source projects such as SymPy, where I worked on improving type annotations and cleaning up legacy code, further enhancing my experience with collaborative development and large codebases.

Some of my open source contributions are listed as follows:

Radis:

- #885 Added parallel CO2 chunk downloader with ThreadPoolExecutor
- #954 Register partial HITEMP CO2 downloads in radis.json
- #977 Fix temporary database downloads being written to working directory
- #853 Replace numpy.trapz with scipy.integrate.trapezoid
- #984 Move profiler out of conditions
- #980 Add warning for out of range wavenumber requests
- #844 Refactor print statements to use f strings

Sympy:

- #28383 Improve typing for as_coeff_Mul using overloads and Literal
- #28484 Remove leftover Python 2 __cmp__ references

Why me:

I want to briefly explain how I approach building systems, because I think it matters for a project like this.

I try to understand things from first principles rather than treating tools as black boxes. While working on this proposal, I didn't just compare libraries based on results, but focused on why they behave differently and what trade offs they make. That is what led me to not only benchmark multiple options, but also build small prototypes for the backend abstraction and cache system instead of stopping at theory.

For me, this project is not just about integrating a new library into RADIS, but about designing something that is maintainable, extensible, and useful in practice. I enjoy working on problems where understanding system behavior and real world constraints matters more than just making something work.

I have already been contributing to RADIS and am familiar with its codebase and workflows, which helps me approach this project with clarity and confidence. I am comfortable iterating based on feedback and working closely with maintainers.

I am genuinely interested in this problem space, and I intend to continue contributing to RADIS beyond GSoC and improving the system over time.

Availability to contribute to the project:

I have structured my schedule to prioritize this project throughout the GSoC period. Since this aligns with my semester break, I will be able to focus on development without major academic commitments. My university resumes in mid-July, but the initial weeks are relatively light, which allows me to continue contributing consistently and stay on track with the project timeline.

About Me:

I am a sophomore at the Indian Institute of Information Technology, Lucknow majoring in Computer Science. I began my journey in tech through competitive programming, which helped me build a strong foundation in problem solving and algorithms (LeetCode, Codeforces). I secured AIR 19 in the India International Women's Competitive Programming Championship and was part of a 1st Runner-Up team in the Uber Hackathon, where we had the opportunity to present our solution at the Uber Bangalore office in front of experienced engineers.

As I started working on projects during my early years of engineering, I realized that writing code is only one part of software development. Maintaining and improving existing systems is equally important. This realization led me to open source. Since I was already comfortable with Python, I began contributing to Python based libraries, which introduced me to collaborative development and working with large codebases.

Alongside this, I have built projects such as SSrename (link), an OCR based screenshot renaming tool, along with other systems including a CLI tool and a rendering engine.

Through this journey, I came across RADIS and have been actively contributing to it. The supportive nature of the maintainers during my initial contributions motivated me to stay involved and deepen my understanding of the project.

I am excited about the opportunity to work on this project, as it aligns closely with my interests in performance optimization, data processing, and real world system design. I am committed to contributing meaningfully and continuing my involvement with RADIS beyond the GSoC period.

Contact Information:

Name: Merin Theres Jose

GitHub: https://github.com/merintheres

Contact Number: +91 - 8590396998

LinkedIn: https://www.linkedin.com/in/merintheresjose

University: Indian Institute of Information Technology, Lucknow (https://iiitl.ac.in/)

Location: (GMT+05:30) Asia/India – IST